

National Textile University, Faisalabad



Department of Computer Science

Name:	Zainab sajid
Class:	BS (SE) – 5B
Registration No:	23-NTU-CS-FL-1218
Assignment:	Assignment 1
Course Code:	DAA
Course Name:	Design and Analysis of Algorithm
Submitted To:	Sir Hamid
SubmissionDate:	9-OctS-2025

Assignment No. 01: Algorithm Writing Styles

BSSE 5B

Fall 2025

Due Date: October 12, 2025

Instructions:

Answer all questions clearly and concisely. For algorithm-related tasks, include both pseudocode and, if applicable, a flowchart. Submit your answers in PDF format.

Q. 1: Explain what pseudocode is and why it is used. How does it differ from a formal programming language?

Pseudocode

1. Definition

Pseudocode is a method of designing algorithms using plain, simple, and structured English statements.

It describes the steps or logic needed to solve a problem without worrying about programming syntax or specific language rules.

Pseudocode acts as a bridge between human thinking and computer programming, helping programmers and non-programmers alike to understand how a program will work.

2. Purpose of Using Pseudocode

Pseudocode is used mainly for planning, understanding, and communication during software development.

Here are the key reasons why it is important:

1. To Plan a Program Before Coding

- Writing pseudocode helps programmers outline the program structure and flow of logic before starting to code.
- It reduces errors and confusion during the coding stage.

2. To Focus on Logic Instead of Syntax

- When writing pseudocode, the writer does not worry about specific language rules (like semicolons or data types).
- This allows full focus on the logical sequence of steps to solve a problem.

3. To Communicate and Explain Algorithms Clearly

- Pseudocode is written in plain English, so even people without programming experience can understand how the algorithm works.
- It is often used in team discussions, documentation, and algorithm design.

4. To Avoid Syntax Errors

- Since pseudocode does not follow strict rules of a programming language, it eliminates syntax-related mistakes that can occur in actual coding.

4. Difference Between Pseudocode and Programming Language

Feature	Pseudocode	Programming Language
Syntax	Informal, English-like instructions	Follows strict and formal grammar/syntax rules
Execution	Cannot be executed by a computer	Can be compiled or interpreted and executed
Purpose	Used to design, explain, and plan algorithms	Used to implement and run actual programs
Readability	Very easy to read and understand for everyone	Requires programming knowledge to read
Flexibility	Can be adapted easily to any language	Specific to one programming language (e.g., Python, Java, C++)
Error Handling	No syntax errors possible	Syntax and logic errors can occur
Output	Used only for planning; no output generated	Produces actual results or outputs when executed

. Example of Pseudocode

Example : Find the Largest of Two Numbers

BEGIN

INPUT number1, number2

IF number1 > number2 THEN

```
    PRINT "Number1 is greater"
ELSE
    PRINT "Number2 is greater"
ENDIF
END
```

Q. 2: List and explain at least three advantages of using pseudocode to describe algorithms before coding.

1. Easy to Understand (Clarity)

Pseudocode is written in simple, English-like statements, which makes it easy to understand even for those who are not programmers.

It helps in presenting the logic of the program step by step without worrying about syntax or programming rules.

Because of its readability, both beginners and experienced developers can clearly follow how the algorithm works before writing the actual code.

Example:

```
BEGIN
INPUT marks
IF marks >= 50 THEN
PRINT "Pass"
ELSE
PRINT "Fail"
ENDIF
END
```

This pseudocode clearly shows the logic of checking whether a student has passed or failed, without needing to know any programming language.

2. Detects Errors Early (Error Reduction)

Writing pseudocode helps in identifying and fixing logical errors before coding begins.

Since it focuses purely on logic, developers can easily test whether the sequence of steps will solve the problem correctly.

By catching mistakes early, pseudocode saves time and effort that would otherwise be spent debugging after writing the actual code.

Example:

If the goal is to find the largest of three numbers, pseudocode helps verify the logic before coding:

```
BEGIN  
INPUT A, B, C  
IF A > B AND A > C THEN  
PRINT "A is largest"  
ELSE IF B > C THEN  
PRINT "B is largest"  
ELSE  
PRINT "C is largest"  
ENDIF  
END
```

Here, you can easily check if all conditions are correct before writing it in Python, C++, or Java.

3. Improves Communication

Pseudocode serves as a common language between programmers, designers, and even non-technical team members.

It allows everyone involved in a project to discuss and understand the algorithm without needing to know a specific programming language.

This improves collaboration during planning, documentation, and review stages.

Example:

```
BEGIN  
INPUT username, password  
IF username and password match stored data THEN  
    PRINT "Login Successful"  
ELSE  
    PRINT "Invalid Login"  
ENDIF  
END
```

Even a non-technical manager or client can understand this logic and confirm that it works as intended.

Q. 3: Compare and contrast pseudocode and flowcharts. In what scenarios would you prefer one over the other?

Comparison between Pseudocode and Flowcharts

Feature	Pseudocode	Flowchart
Form	Written using simple English-like statements	Diagrammatic representation using symbols and arrows
Focus	Focuses on the logical steps of the algorithm	Focuses on the flow and sequence of operations
Ease of Writing	Quick and easy to write and modify	Takes more time to draw and update
Understanding	Easier for people comfortable with text	Easier for visual learners to understand process flow
Detail Level	Describes logic step by step	Shows overall structure and flow of control

When to Use Pseudocode

When You Want to Plan Logic Quickly Before Actual Coding

Pseudocode is useful in the early stage of program development when you need to plan the logic of a program step by step.

It allows you to focus on the problem-solving process without worrying about programming syntax.

This helps in saving time and organizing the program's structure clearly before implementation.

2. When the Algorithm is Complex and Requires Step-by-Step Explanation

For complicated algorithms, pseudocode helps in breaking down the process into smaller, understandable steps.

It provides a clear outline of the operations to be performed, making it easier to understand and later convert into code.

This ensures that all parts of the algorithm are well-planned and logically sound.

3. When Working in a Text-Based Environment (Documentation or Design Notes)

Pseudocode is suitable when preparing written reports, documentation, or design materials.

It fits well in text-based formats and can be easily shared among team members.

It helps communicate program logic clearly without the need for diagrams or graphics.

When to Use Flowcharts

1. When You Need a Visual Representation of the Program's Flow

Flowcharts are helpful for showing how a program progresses from one step to another visually.

They make the flow of control easier to understand through the use of shapes and arrows.

This visual layout helps in identifying the overall structure of a program at a glance.

2. When Explaining Logic to Non-Technical Users or Beginners

Flowcharts are ideal for presentations or explanations to people who are not familiar with coding.

The visual symbols make it easier for them to understand how the program works.

It simplifies communication between technical and non-technical audiences.

3. When Identifying Decision Points and Loops Visually in a Process

Flowcharts clearly show where decisions and loops occur in a program.

This helps in analyzing the logic, spotting possible errors, and improving efficiency.

It is especially useful during the design and debugging stages of program development.

Q. 4: Write pseudocode for a **linear search algorithm** that finds a specific value in an array of integers. Clearly label the input, process, and output.

Pseudocode

Input:

- An array of integers ARR[1..N]
- A number TARGET to search

Process:

SET found = false

FOR i = 1 TO N DO

 IF ARR[i] = TARGET THEN

 SET found ← true

 SET position ← i

 END IF

END FOR

Output:

IF found = true THEN

 PRINT "Element found at position", position

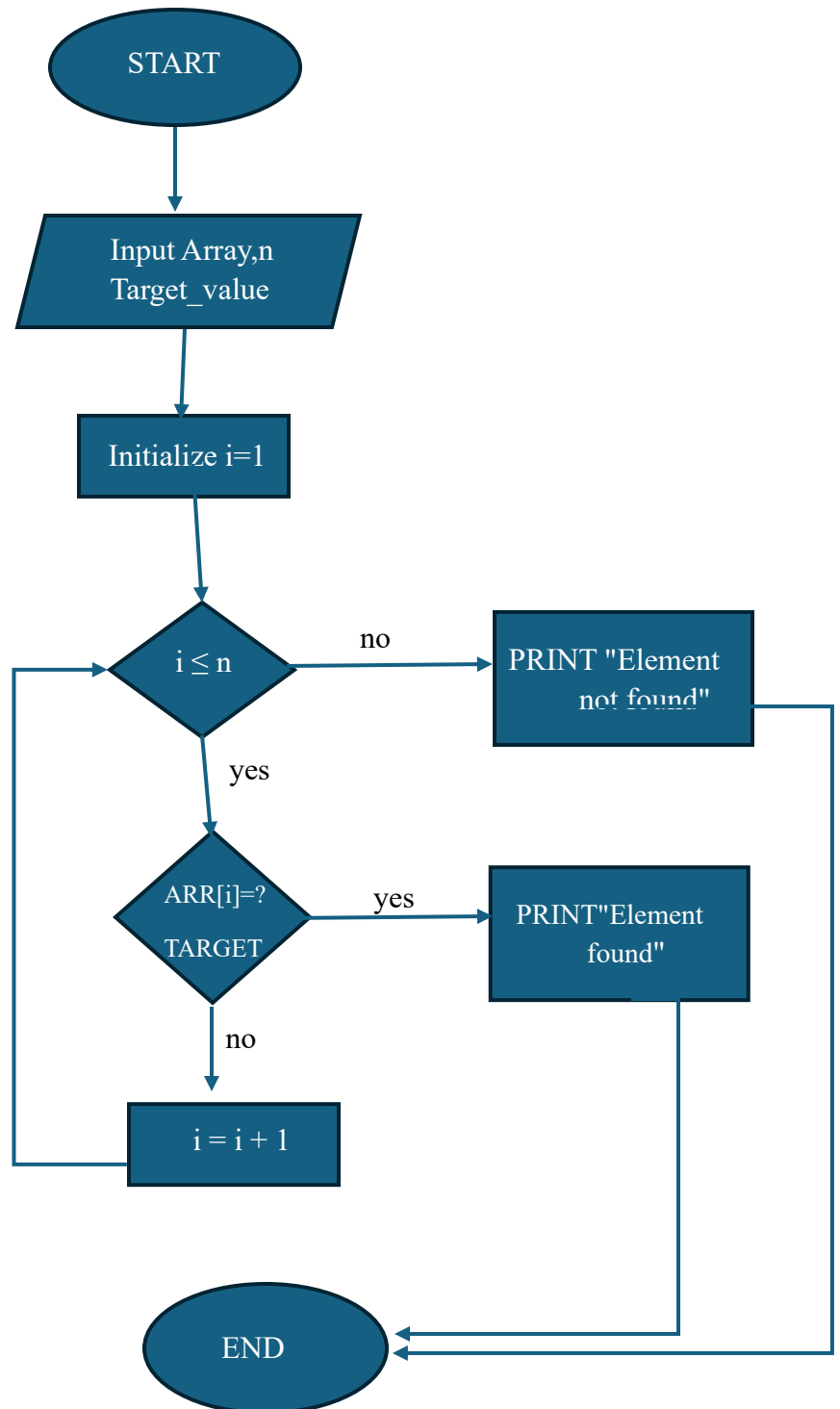
ELSE

 PRINT "Element not found"

END IF

Q. 5: Draw a flowchart for the **linear search algorithm** described in question 4. Include all relevant symbols to depict input, output, and decision-making.

Flowchart



Q. 6: Write pseudocode that checks whether a given number is positive, negative, or zero, and prints the corresponding message.

Pseudocode

Input: number

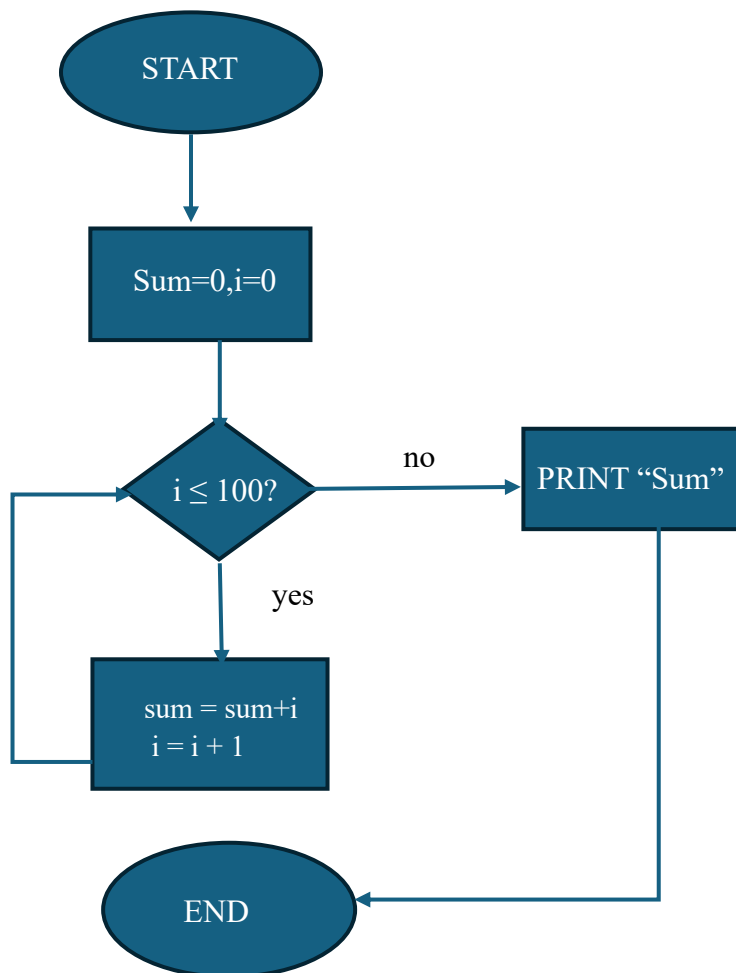
Process:

```
IF number > 0 THEN
    PRINT "The number is positive"
ELSE IF number < 0 THEN
    PRINT "The number is negative"
ELSE
    PRINT "The number is zero"
END IF
```

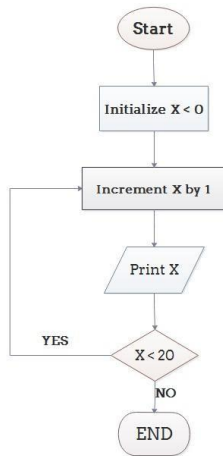
Output: Message showing result

Q. 7: Design a flowchart for an algorithm that sums all numbers from 1 to 100. Include decisionmaking for the looping process.

Flowchart



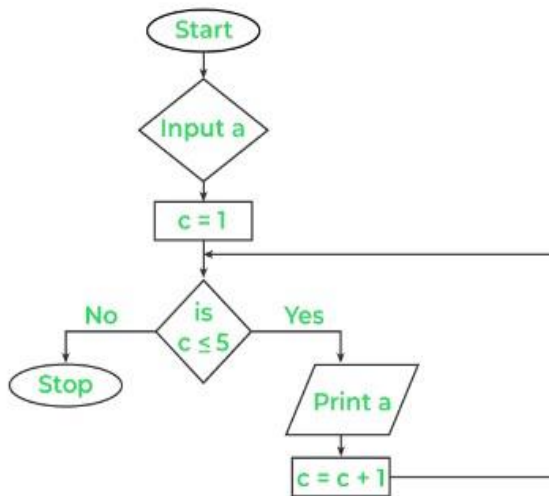
Q. 8: Below is a flowchart: Write the pseudocode that corresponds to this flowchart.



Pseudocode:

1. Start
2. Initialize variable **X = 0**
3. Repeat the following steps:
 - a. Increment **X** by 1
 - b. Print the value of **X**
 - c. Check if **X < 20**
 - If **YES**, go back to step 3
 - If **NO**, proceed to the next step
4. End

Q. 9: Below is a flowchart. Write the pseudocode that corresponds to this flowchart.



Pseudocode:

1. Start
2. Input the value of **a**
3. Set **c = 1**
4. Check if **c ≤ 5**
 - a. If **Yes**, then:
 - i. Print the value of **a**
 - ii. Increment **c** by 1 (**c = c + 1**)
 - iii. Go back to step 4
 - b. If **No**, go to step 5
5. Stop

Q. 10: Write pseudocode for a **bubble sort algorithm** that sorts a list of numbers in ascending order. Explain each step in your pseudocode.

Pseudocode: Bubble Sort

1. Start
2. Input the number of elements, **n**
3. Input **n** elements into the array **A**
4. For **i = 0** to **n - 2**, repeat step 5
 5. For **j = 0** to **n - i - 2**, do
 - a. If **A[j] > A[j + 1]**, then
 - i. Swap **A[j]** and **A[j + 1]**
5. End inner loop
6. End outer loop
7. Print the sorted array **A**
8. Stop

Q. 11: Discuss the strengths and weaknesses of both pseudocode and flowcharts in algorithm design. In your opinion, which is more useful for beginners and why?

Pseudocode

Pseudocode is a **text-based description** of an algorithm written in a format that resembles programming languages but without strict syntax rules. It focuses on the **logic and steps** of the program rather than exact code.

Strengths of Pseudocode:

1. **Easy to Write and Understand:**

It uses plain English-like statements, so it's simple for anyone with basic logic skills to follow.

2. **Close to Real Programming:**

Pseudocode helps students or developers think like programmers since it uses structures such as *IF-THEN-ELSE*, *FOR*, *WHILE*, etc.

3. **Fast to Create and Modify:**

Writing pseudocode is quicker than drawing flowcharts and easier to update when changes are needed.

4. **Useful for Complex Algorithms:**

It handles complex conditions, loops, and data structures more easily than flowcharts.

Weaknesses of Pseudocode:

1. **No Standard Format:**

There's no universal rule for writing pseudocode different people may write it differently, causing confusion.

2. **Less Visual:**

Since it's text-based, it may be harder for visual learners to understand the logic flow compared to diagrams.

3. **Hard to Spot Logical Mistakes Quickly:**

Errors in logic might not be as easily visible as they are in flowcharts.

Flowcharts

A flowchart is a **diagrammatic representation** of an algorithm using symbols like ovals, rectangles, diamonds, and arrows to show the flow of control.

Strengths of Flowcharts:

1. **Visual Clarity:**

It shows the entire process graphically, making it easy to follow and understand the program's flow at a glance.

2. **Good for Communication:**

Flowcharts are excellent for explaining algorithms to non-programmers, managers, or beginners.

3. **Error Detection:**

Logical errors or missing steps can often be spotted easily because the flow is visual.

4. **Documentation Tool:**

Flowcharts serve as a clear visual document for system design and process explanation.

Weaknesses of Flowcharts:

1. **Time-Consuming:**

Drawing detailed flowcharts for large programs takes significant time and effort.

2. **Difficult to Modify:**

Any small change in logic may require redrawing a large portion of the chart.

3. **Not Suitable for Complex Programs:**

For algorithms involving many conditions, loops, or functions, flowcharts can become cluttered and confusing.

4. **Requires Symbols Knowledge:**

Beginners need to learn standard symbols (like decision diamonds, process rectangles, etc.) before they can create or read flowcharts.

Which Is More Useful for Beginners and Why?

In my opinion, **flowcharts are more useful for absolute beginners, while pseudocode becomes more helpful once they understand basic programming logic.**

Reasons:

- **Flowcharts for Beginners:**

- ✓ They provide a clear **visual understanding** of how decisions and loops work.
- ✓ New learners can easily see the **flow of control** and understand what happens step by step.

- ✓ It helps them develop **logical thinking** before worrying about coding syntax.
- **Pseudocode for Intermediate Learners:**
 - ✓ Once a student understands the logic visually, pseudocode helps them **transition smoothly into real coding**.
 - ✓ It allows focus on **logic construction** using terms similar to programming languages